Unit 2 - Search Algorithms

Artificial Intelligence | Unit 2

Introduction to Search Algorithms

Search algorithms are techniques used by AI agents to find solutions to problems by exploring a state space. The agent starts from an initial state and uses a search strategy to reach a goal state by applying a series of actions.

Types of Search

Uninformed Search (Blind Search)

No additional information about states beyond the problem definition. Examples: BFS, DFS, Uniform Cost Search.

Informed Search (Heuristic Search)

Uses problem-specific knowledge to find solutions more efficiently. Examples: A* Search, Greedy Best-First Search.

Breadth First Search (BFS)

- Explores all nodes at current depth before going deeper
- Uses a Queue (FIFO) data structure
- Guarantees shortest path in unweighted graphs
- Time complexity: O(b^d) where b=branching factor, d=depth
- Space complexity: O(b^d)

Depth First Search (DFS)

- Explores as deep as possible before backtracking
- Uses a Stack (LIFO) data structure
- Does not guarantee shortest path
- Time complexity: O(b^m) where m=maximum depth

- Space complexity: O(bm) - much better than BFS

Heuristic Search - Hill Climbing

Hill Climbing is a local search algorithm that continuously moves in the direction of increasing value to find the peak. It is greedy and never backtracks, making it fast but susceptible to local maxima, plateaus, and ridges.

Simulated Annealing

Inspired by metallurgy annealing process. Accepts worse solutions with a probability that decreases over time (temperature). This allows the algorithm to escape local optima. The acceptance probability is e^(delta/T) where T is the current temperature.

A* Algorithm

A* uses f(n) = g(n) + h(n) where g(n) is the cost from start to node n, and h(n) is the estimated cost from n to goal (heuristic). It is complete and optimal when the heuristic is admissible (never overestimates).